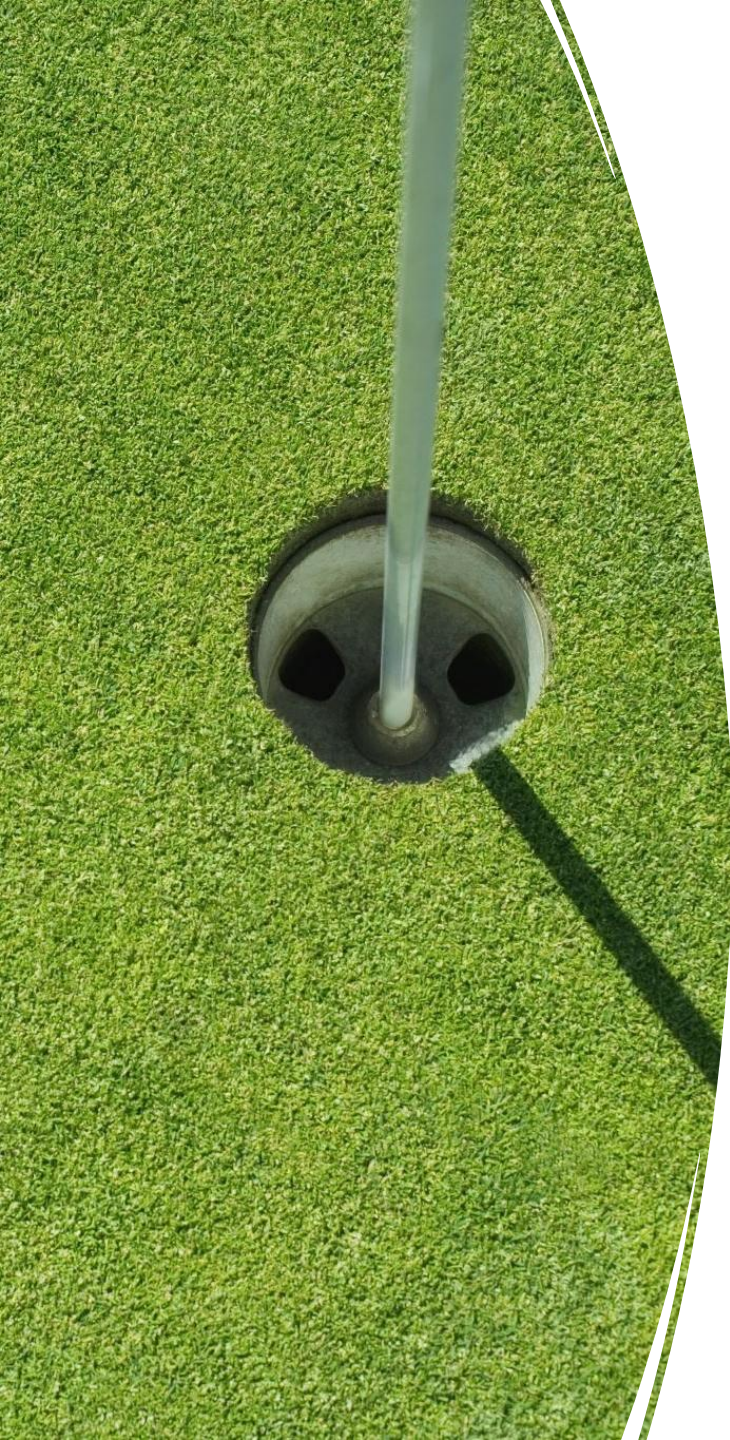




Course Review (2)

Context-Free Language

Use context-free grammar to generate the context-free language

Each production rule has this form: $A \rightarrow \alpha, \alpha \in (V \cup T)^*$

Leftmost/Rightmost derivation, parse tree, ambiguous grammar.

Recursive rules: left/right recursion, center recursion

s-grammar: $A \rightarrow a\alpha, \alpha \in V^*$

s-grammar: rules with the same left side must start with different terminals.

CFG Simplification

After that, the language becomes lambda-free

Suggested order: remove lambda-productions, remove unit-production, remove useless productions

Chomsky normal form: $A \rightarrow BC$, $A \rightarrow a$

Greibach normal form: $A \rightarrow a\alpha$, $\alpha \in V^*$

Greibach normal form: substitute the leading variable or change the left recursion to right recursion

Pushdown Automata

Machine $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$

Stack-based storage: pop one symbol and push several symbols at one move

Use to recognize context-free languages

Accept by final state, regardless of the stack content

npda vs. dpda (having lambda- and no-moves)

CFG to NPDA Conversion

—

- Construct three-state transition diagram
- Push the grammar start symbol into the stack:
 $\delta(q_0, \lambda, z) = (q_1, Sz)$
- Translate each production rule into an edge transition
- $A \rightarrow \alpha \Rightarrow \delta(q_1, \lambda, A) = (q_1, \alpha)$
- For every terminal a in α , add: $\delta(q_1, a, a) = (q_1, \lambda)$

NPDA to CFG Conversion

Make sure all transitions are one of two forms

$$\delta(q_i, a, A) = (q_j, \lambda) \text{ into } A_{ij} \rightarrow a$$

$$\delta(q_i, a, A) = (q_j, BC) \text{ into } A_{ik} \rightarrow aB_{jl}C_{lk}$$

z_{of} is the grammar start symbol, if q_f is the final state